

XGBoost 深度学习手册：数学原理与时间序列分析应用详解

1. 执行摘要与引言

XGBoost (Extreme Gradient Boosting, 极限梯度提升) 作为一种可扩展的端到端树提升系统, 已成为机器学习领域的基石算法。尽管其最初的设计初衷是解决静态表格数据的分类与回归问题, 但凭借其卓越的泛化能力、系统级优化以及对非线性关系的强大捕捉能力, XGBoost 在时间序列预测领域也展现出了超越传统统计模型 (如 ARIMA) 和部分深度学习模型的潜力¹。

本研究报告旨在提供一份详尽的专家级指南, 深入剖析 XGBoost 的数学核心, 并系统性地阐述其在时间序列分析中的应用范式。报告将从二阶泰勒展开的目标函数推导出发, 探讨稀疏感知分裂算法的机制, 进而深入时间序列的特征工程、非平稳数据处理、多步预测策略 (递归与直接法) 以及严格的回测验证协议。通过整合最新的研究成果与工程实践, 本文档将揭示如何通过特征工程克服树模型无法外推 (Extrapolation) 的本质缺陷, 以及利用 GPU 加速和分布式计算处理大规模时序数据的最佳实践²。

2. XGBoost 数学原理与核心架构

要精通 XGBoost 在复杂场景下的应用, 必须首先理解其底层的数学构建。与传统的梯度提升机 (GBM) 不同, XGBoost 在目标函数的近似、正则化项的设计以及树的生长策略上进行了根本性的改进。

2.1 梯度提升决策树 (GBDT) 的加法模型

XGBoost 属于集成学习中的 Boosting 家族, 其核心思想是通过迭代地训练弱学习器 (通常是 CART 回归树) 来拟合前一轮模型的残差。对于一个包含 n 个样本和 m 个特征的数据集 $\mathcal{D} = \{(x_i, y_i)\} (|\mathcal{D}| = n, x_i \in \mathbb{R}^m, y_i \in \mathbb{R})$, 模型在第 t 轮的预测值 $\hat{y}_i^{(t)}$ 可以表示为前一轮预测值与当前树 $f_t(x_i)$ 的加和:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i), \quad f_t \in \mathcal{F}$$

其中 $\mathcal{F} = \{f(x) = w_{q(x)}\}$ 是回归树的空间, $q: \mathbb{R}^m \rightarrow T$ 表示将样本映射到叶子节点的结构函数, $w \in \mathbb{R}^T$ 是叶子节点的权重向量, T 是叶子节点的数量¹。

这种加法训练策略 (Additive Training) 将复杂的优化问题分解为多个简单的子问题。在每一步中

，算法并不直接优化整个集成模型，而是贪婪地寻找一个最优的函数 f_t 来最大限度地降低目标函数。

2.2 目标函数的二阶泰勒展开

XGBoost 的数学精髓在于其对目标函数的处理。传统的 GBM 通常仅利用损失函数的一阶导数（梯度）来指导学习，而 XGBoost 引入了二阶导数（黑塞矩阵，Hessian），这使得收敛速度更快，且能更精确地逼近损失函数的极值点。

第 t 轮的目标函数 $\mathcal{L}^{(t)}$ 由训练误差和正则化项两部分组成：

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

其中 l 是可微凸损失函数（如均方误差 MSE 或 Log Loss）， $\Omega(f_t)$ 是控制模型复杂度的正则化项。为了优化该目标函数，XGBoost 对损失函数在 $\hat{y}_i^{(t-1)}$ 处进行二阶泰勒展开¹：

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

在此公式中， g_i 和 h_i 分别是一阶和二阶梯度统计量：

$$g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)}), \quad h_i = \partial_{\hat{y}^{(t-1)}}^2 l(y_i, \hat{y}^{(t-1)})$$

移除常数项 $l(y_i, \hat{y}_i^{(t-1)})$ 后，简化的目标函数变为：

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

这一形式的优越性在于它将损失函数的具体形式与优化算法解耦。只要损失函数是二次可微的，XGBoost 的核心算法即可通用，这为定义自定义目标函数（如用于时间序列的 Quantile Loss）提供了坚实的数学基础⁶。

2.3 正则化与结构分数

为了防止过拟合，XGBoost 在目标函数中显式加入了正则化项 $\Omega(f)$ 。这是其区别于传统 GBM 的关键特性之一。正则化项定义为：

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

- **γ (Gamma):** 叶子节点分裂的最小损失减少阈值。这相当于树的预剪枝参数。只有当分裂带来的增益大于 γ 时，节点才会分裂。
- **λ (Lambda):** 叶子节点权重的 L2 正则化系数。这有助于平滑叶子节点的预测值，避免个别叶子节点权重过大导致的过拟合¹。

将正则化项代入目标函数，并根据叶子节点 j 对样本进行重组（定义 $I_j = \{i | q(x_i) = j\}$ 为落入叶子节点 j 的样本集合），我们可以推导出最优叶子节点权重 w_j^* 和对应的最优目标函数值（结构分数）：

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

$$\tilde{\mathcal{L}}^* = - \frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

公式 $\tilde{\mathcal{L}}^*$ 被称为**结构分数 (Structure Score)**，它类似于决策树中的基尼系数或熵，用于评估树结构的质量。分数越低（代数上越小），结构越优¹。

2.4 分裂查找算法：从贪婪到近似

在构建树的过程中，核心挑战是如何找到最优的特征分裂点。

1. 精确贪婪算法 (Exact Greedy Algorithm):

对于每个特征，算法会对所有样本的特征值进行排序，并遍历所有可能的切分点，计算分裂后的增益 (Gain)：

$$\text{Gain} = \frac{1}{2} \left(\sum_{i \in I_L} h_i - \sum_{i \in I_R} h_i \right)^2$$

这种方法在数据量较小时非常精确，但在大数据集下计算成本极高且无法适应内存限制⁸。

2. 加权分位数略图 (Weighted Quantile Sketch) 与近似算法：

为了解决效率问题，XGBoost 引入了近似算法。它根据特征分布的百分位数（分位数）提出候选切分点。由于二阶梯度

h_i 代表了样本在损失函数中的曲率（权重），XGBoost 使用加权分位数略图来选择候选点，使得候选点能够均匀地分割数据的权重，而不仅仅是样本数量。这保证了在数据分布不均时仍能找到高质量的切分点²。

3. 稀疏感知分裂 (Sparsity-aware Split Finding):

现实世界的时序数据往往包含缺失值（如传感器故障）。XGBoost 能够自动学习缺失值的默认分裂方向。在每个节点，算法分别尝试将缺失值归入左子树和右子树，选择增益最大的方向作为该节点的默认方向。这意味着用户

无需手动填充缺失值，模型能够利用缺失本身作为一种信息¹⁰。

3. 时间序列分析中的概念范式转移

将 XGBoost 应用于时间序列预测，需要打破传统回归模型中样本独立同分布（IID）的假设。时间序列数据具有内在的时间依赖性和自相关性，因此必须通过特定的数据转换技术，将时序问题重构为监督学习问题。

3.1 监督学习转换与滑动窗口机制

XGBoost 无法直接处理原始的时间序列序列（如 $y_1, y_2, \dots, y_t$ ）。必须通过**滑动窗口（Sliding Window）**技术，将序列转换为输入特征矩阵 X 和目标向量 y ¹²。

该过程的核心在于利用历史时间步（Lags）来预测未来时间步。对于给定的时间点 t ，我们构建特征如下：

样本索引	目标 (yt)	特征 1 (yt-1)	特征 2 (yt-2)	...	特征 n (yt-n)
1	y_{n+1}	y_n	y_{n-1}	...	y_1
2	y_{n+2}	y_{n+1}	y_n	...	y_2

这种重构将时序依赖转化为特征列之间的关系。在 Python 中，这通常通过 Pandas 的 shift() 函数实现¹³。

3.2 树模型的致命弱点：外推性（Extrapolation）

在应用 XGBoost 进行时间序列预测时，最关键的限制在于**树模型无法外推趋势**。

- **数学解释：**决策树的本质是将特征空间划分为互不重叠的矩形区域，并在每个区域内输出一个常数（叶子节点的权重 w ）。因此，整个模型是一个分段常数函数（Piecewise Constant Function）。
- **时序后果：**如果训练数据中的目标值 y 范围在 $[\$ \$]$ 之间，且存在明显的线性增长趋势，当模型遇到时间特征 t 对应的未来数据时，它只能预测出训练集中见过的最大值（如 100 左右），而无法预测出 110 或 120。模型在特征空间之外的预测值将坍塌为常数³。

应对策略：

这是使用 XGBoost 处理带趋势时序数据的核心难点。解决方案通常包括：

1. **差分 (Differencing)**：预测 $y_t - y_{t-1}$ 而非 y_t 。
2. **去趋势 (Detrending)**：使用线性模型拟合趋势，XGBoost 预测残差。
3. **线性树 (Linear Tree)**：在叶子节点中使用线性回归而非在常数，这是部分 GBM 库（如 LightGBM）的高级特性，XGBoost 在较新版本中也有实验性支持³。

4. 时间序列的高级特征工程

由于 XGBoost 缺乏内建的时序理解能力，特征工程的质量直接决定了预测的上限。所有的时间依赖性都必须显式地编码为特征。

4.1 滞后特征 (Lag Features)

这是最基础也是最重要的特征。除了直接的一阶滞后 ($t - 1$)，还应构建：

- **季节性滞后**：对于月度数据，引入 $t - 12$ ；对于小时数据，引入 $t - 24$ 。这能让模型直接“看见”上一周期的状态。
- **长短期记忆**：结合短期滞后 ($t - 1, t - 2$) 和长期滞后 ($t - 30$) 以捕捉不同尺度的依赖¹²。

4.2 滚动窗口统计量 (Rolling Window Statistics)

为了捕捉序列的局部稳定性或波动性，需要计算滑动窗口内的统计指标：

- **滚动均值 (Rolling Mean)**： $\frac{1}{w} \sum_{i=1}^w y_{t-i}$ ，用于平滑噪声，捕捉局部趋势。
- **滚动标准差 (Rolling Std)**：衡量最近的波动率或不确定性。
- **滚动最值 (Min/Max)**：捕捉局部的极值范围。
- **扩展窗口 (Expanding Window)**：窗口大小随时间增加，计算截止当前的所有历史数据的统计量¹²。

4.3 时间编码与循环特征

简单地将“月份”编码为 1-12 的整数对于树模型虽然可行（树可以切割 $\text{Month} \leq 6$ ），但无法表达 12 月与 1 月的邻近性。更优的做法是**循环编码 (Cyclical Encoding)**¹⁴：

$$x_{\sin} = \sin\left(\frac{2\pi \cdot t}{\text{max_period}}\right), \quad x_{\cos} = \cos\left(\frac{2\pi \cdot t}{\text{max_period}}\right)$$

例如，对于小时特征（0-23）， $\text{max_period} = 24$ 。这种编码将时间映射到单位圆上，保留了时间的周期连续性。

5. 处理非平稳性与趋势

非平稳性（Non-stationarity）是指时间序列的统计特性（均值、方差）随时间变化。XGBoost 对非平稳数据的处理能力较弱，必须进行预处理。

5.1 差分技术（Differencing）

差分是消除趋势和季节性的标准方法。

- 一阶差分： $y'_t = y_t - y_{t-1}$ 。这可以将线性趋势转化为围绕 0 波动的平稳序列。XGBoost 学习的是“变化的量”。预测后，需要通过累加（Cumulative Sum）还原预测值：

$$\hat{y}_t = \hat{y}'_t + y_{t-1}$$

- 季节性差分： $y'_t = y_t - y_{t-s}$ （例如 $s = 12$ ）。用于消除季节性模式。

5.2 对数变换（Log Transformation）

对于方差随时间增大的序列（例如指数增长的销售额），对数变换 $z_t = \ln(y_t)$ 可以稳定方差，将乘性模式转化为加性模式，使其更适合加法模型（如 XGBoost）。预测结束后需使用指数函数还原： $\hat{y}_t = \exp(\hat{z}_t)$ 。

5.3 混合模型策略（Hybrid Modeling）

一种强大的工业界实践是“线性模型 + XGBoost”的混合架构：

- 使用线性回归或 ARIMA 拟合序列的**趋势**部分。
- 计算残差： $r_t = y_t - \hat{y}_{\text{trend}}$ 。
- 训练 XGBoost 模型预测残差 r_t ，输入为各种滞后特征和外生变量。
- 最终预测： $\hat{y}_{\text{final}} = \hat{y}_{\text{trend}} + \hat{r}_{\text{xgb}}$ 。

这种方法结合了线性模型的外推能力和 XGBoost 拟合非线性残差的强大能力³。

6. 多步预测策略架构

单步预测 ($t + 1$) 相对简单,但在实际业务中,往往需要预测未来多个时间步 ($t + 1, \dots, t + h$)。XGBoost 支持多种策略,每种策略在计算成本和误差传播上各有优劣。

6.1 递归策略 (Recursive Strategy)

训练一个单一模型 f 用于预测 $t + 1$ 。在预测 $t + 2$ 时,将模型对 $t + 1$ 的预测值作为输入特征喂回模型。

- **优点:** 只需训练一个模型,计算资源消耗最低。
- **缺点: 误差累积 (Error Accumulation)。** $t + 1$ 的预测误差会传播到 $t + 2$, 导致长期预测精度迅速下降¹⁸。

6.2 直接策略 (Direct Strategy)

为预测视界中的每一步训练一个独立的模型。模型 f_1 预测 $t + 1$, 模型 f_2 预测 $t + 2$, 以此类推。

- **优点:** 无误差累积。每个模型专门针对其预测跨度进行优化 (例如 f_{12} 可以专门学习年度季节性)。
- **缺点:** 计算成本高 (需训练 h 个模型)。忽略了目标变量之间的依赖关系 (y_{t+1} 和 y_{t+2} 通常是相关的,但独立模型无法捕捉这种相关性)²⁰。

6.3 DirRec 策略 (Hybrid)

结合递归和直接策略。为每一步训练单独的模型,但后续的模型将前一步模型的预测值作为新的特征加入输入集。

- **机制:**
 - $f_1(X) \rightarrow \hat{y}_{t+1}$
 - $f_2(X, \hat{y}_{t+1}) \rightarrow \hat{y}_{t+2}$
- **特点:** 试图在捕捉依赖关系和特定步长优化之间取得平衡,但模型输入维度会随预测步长增加而膨胀²²。

6.4 多输出策略 (Multi-Output / Vector Leaf)

利用 Scikit-Learn 的 MultiOutputRegressor 或 XGBoost 原生的多输出支持。

- **Scikit-Learn 包装器:** 本质上是直接策略的自动化封装,训练 h 个回归器²⁴。
- **XGBoost 原生多输出 (Vector Leaf):** 从 XGBoost 2.0 开始,支持 `multi_strategy='multi_output_tree'`。在这种模式下,树的叶子节点不再是一个标量,而是一

个向量（维度为 h ）。

- **优势**：只需训练一个模型；树结构在构建时会同时考虑所有目标变量的梯度，从而捕捉不同预测步长之间的相关性。这是目前最高效且理论上最优的方案之一⁶。
- **配置**：需设置 `tree_method='hist'` 并指定 `multi_strategy='multi_output_tree'`。

7. 模型评估与验证协议

在时间序列中，随机划分（如 K-Fold Cross Validation）是严格禁止的，因为它会导致**数据泄露（Data Leakage）**，即使用未来的信息预测过去²⁶。

7.1 步行前向验证 (Walk-Forward Validation)

正确的验证方法是基于时间的滚动划分，通常由 `sklearn.model_selection.TimeSeriesSplit` 实现。

- **Fold 1**: Train $[0, t]$, Test $[t + 1, t + k]$
- **Fold 2**: Train $[0, t + k]$, Test $[t + k + 1, t + 2k]$
- **Fold 3**: Train $[0, t + 2k]$, Test $[t + 2k + 1, t + 3k]$

这种方法模拟了现实世界中模型随着新数据到来而不断更新和预测的过程。为了严格防止泄露，训练集和测试集之间可以设置一个 gap（间隙），以模拟数据延迟到位的场景²⁸。

7.2 评估指标

- **RMSE (Root Mean Squared Error)**：对大误差敏感，适合标准回归问题。
- **MAE (Mean Absolute Error)**：对异常值鲁棒性更强。
- **MAPE (Mean Absolute Percentage Error)**：提供百分比误差，业务解释性强，但在真实值为 0 时未定义。
- **Quantile Loss (分位数损失)**：如果需要概率预测（置信区间），应使用分位数目标函数。

例如，训练两个模型分别预测 $q = 0.05$ 和 $q = 0.95$ 的分位数，从而构建 90% 的置信区间³。

8. 超参数调优指南（针对时序数据）

XGBoost 的超参数繁多，针对时序数据的调优策略与传统分类问题有所不同。

表 1：时间序列预测的关键超参数建议

参数名称	别名	作用与时序建议
n_estimators	num_boost_round	树的数量。 建议设置较大的值（如 1000+），并配合 early_stopping_rounds 使用。在时序数据有限的情况下，过多的树容易导致过拟合 ³⁰ 。
eta	learning_rate	学习率。 控制每棵树的贡献。时序预测中建议使用较低的学习率（0.01-0.05），这通常能获得更稳健的泛化能力，但需要更多的树 ⁷ 。
max_depth	-	树深。 控制模型复杂度。时序信号往往包含大量噪声，过深的树（>8）容易拟合高频噪声。建议从浅树（3-6）开始尝试 ³¹ 。
subsample	-	样本采样率。 引入随机性。建议 0.7-0.9。但在极短的时间序列中，过低的采样可能破坏时间连续性。
colsample_bytree	-	特征采样率。 特别是当使用了大量高度相关的滞后特征（Lag 1, Lag 2...）时，降低此参数（0.6-0.8）能强迫模型使用长期滞后，避免过度依赖最近的滞后 ³² 。
reg_alpha	L1	L1 正则化。 对于高维稀疏特征（如大量 One-hot 编码的类别）或大量滞后特征，增加 alpha 有助于特征选择 ¹⁶ 。
monotone_constraints	-	单调约束。 如果业务逻辑规定预测值必须随某特征单调

		递增（如累积销量），必须设置此参数 (1) 或 (-1)，这能显著提高模型在趋势上的物理合理性 ³³ 。
--	--	---

9. 高级实现细节与工程实践

9.1 类别特征的原生处理

在旧版本 XGBoost 中，类别特征（如“商店ID”、“城市”）必须进行 One-Hot 编码。这会导致特征空间稀疏化，不仅消耗内存，还往往导致树模型偏向于切分数值特征而非稀疏的类别特征。

从 1.5 版本开始，XGBoost 支持原生的类别数据处理（enable_categorical=True）。

- **机制：**使用最优分区（Optimal Partitioning）技术，而不是 One-Hot。对于一个有 k 个类别的特征，One-Hot 需要 k 个二值特征，而原生处理可以在一个节点上将 k 个类别划分为两个子集，只需 $O(k \log k)$ 的时间复杂度。
- **优势：**在时间序列中，这对于处理高基数特征（如数千个股票代码或商品ID）尤为重要，能显著提升训练速度和精度³⁵。

9.2 GPU 加速训练

对于高频时序数据（如分钟级金融数据），数据量可能达到数千万行。此时 CPU 训练成为瓶颈。

- **配置：**设置 device='cuda' (XGBoost 2.0+) 或 tree_method='gpu_hist'。
- **原理：**GPU 加速主要用于直方图构建阶段。注意，如果数据集较小，数据从内存传输到显存的开销可能导致 GPU 训练反而比 CPU 慢。通常建议在样本数 > 100k 时启用 GPU⁴。

9.3 缺失值与稀疏性

时间序列数据常因传感器故障或节假日休市出现缺失。XGBoost 的**稀疏感知分裂**机制允许直接输入 NaN。模型会在训练时学习将 NaN 分配到左子节点还是右子节点能带来最大增益。这比简单均值填充或线性插值往往更有效，因为它保留了“缺失”本身所包含的信息（例如，夜间传感器关闭可能意味着数值为 0）¹⁰。

10. 结论

XGBoost 为时间序列预测提供了一个强大且灵活的框架。与传统统计模型相比，它能容纳无限的外生变量并捕捉复杂的非线性交互；与深度学习相比，它在中小规模数据上具有训练速度快、可

解释性强（通过 Feature Importance 和 SHAP 值）的优势。

然而，成功的关键在于充分认识其**非外推性**的本质缺陷，并通过差分、去趋势或混合模型等手段予以规避。结合滑动窗口特征工程、正确的回测验证策略以及针对性的超参数调优，XGBoost 能够在零售销量预测、能源消耗估算、金融市场分析等领域构建出工业级的预测系统。

引用说明：本文档中引用的来源 ID（如 ¹）对应于研究阶段收集的原始资料片段。

引用的著作

1. Introduction to Boosted Trees — xgboost 3.1.1 documentation, 访问时间为 二月 11, 2026, <https://xgboost.readthedocs.io/en/stable/tutorials/model.html>
2. [1603.02754] XGBoost: A Scalable Tree Boosting System - arXiv, 访问时间为 二月 11, 2026, <https://arxiv.org/abs/1603.02754>
3. XGBoost for Time Series: You're gonna need a bigger boat ..., 访问时间为 二月 11, 2026, <https://towardsdatascience.com/xgboost-for-time-series-youre-gonna-need-a-bigger-boat-9d329efa6814/>
4. XGBoost GPU Support — xgboost 3.2.0 documentation, 访问时间为 二月 11, 2026, <https://xgboost.readthedocs.io/en/stable/gpu/index.html>
5. XGBoost Mathematics Explained. A walk-through of the Gradient Boosted... | by Dimitris Leventis | Medium, 访问时间为 二月 11, 2026, <https://dimleve.medium.com/xgboost-mathematics-explained-58262530904a>
6. A demo for multi-output regression — xgboost 3.2.0-dev ..., 访问时间为 二月 11, 2026, https://xgboost.readthedocs.io/en/stable/python/examples/multioutput_regression.html
7. XGBoost Parameters — xgboost 3.2.0 documentation, 访问时间为 二月 11, 2026, <https://xgboost.readthedocs.io/en/stable/parameter.html>
8. XGBoost - GeeksforGeeks, 访问时间为 二月 11, 2026, <https://www.geeksforgeeks.org/machine-learning/xgboost/>
9. XGBoost GPU Support — xgboost 0.72.1 documentation, 访问时间为 二月 11, 2026, https://xgboost.readthedocs.io/en/release_0.72/gpu/
10. How Does XGBoost Handle Missing Data? What Are the Drawbacks of This Approach? An Advanced Technique for Handling Missing Values. | by Bakhtiyarsalah | CodeX | Medium, 访问时间为 二月 11, 2026, <https://medium.com/codex/how-does-xgboost-handle-missing-data-305861d2cc82>
11. How does Xgboost learn what are the inputs for missing values?, 访问时间为 二月 11, 2026, <https://datascience.stackexchange.com/questions/15305/how-does-xgboost-learn-what-are-the-inputs-for-missing-values>
12. Time Series forecasting XGBoost:Lags and Rolling - Kaggle, 访问时间为 二月 11, 2026,

- <https://www.kaggle.com/code/sayedathar11/time-series-forecasting-xgboost-lags-and-rolling>
13. How to Use XGBoost for Time Series Forecasting - MachineLearningMastery.com, 访问时间为 二月 11, 2026,
<https://machinelearningmastery.com/xgboost-for-time-series-forecasting/>
 14. [D] How does xgboost work with time series? : r/MachineLearning - Reddit, 访问时间为 二月 11, 2026,
https://www.reddit.com/r/MachineLearning/comments/1aoo7gc/d_how_does_xgb_oost_work_with_time_series/
 15. Time series patient visits for XGBoost classifier - AI Stack Exchange, 访问时间为 二月 11, 2026,
[https://ai.stackexchange.com/questions/48629/time-series-patient-visits-for-xgb_oost-classifier](https://ai.stackexchange.com/questions/48629/time-series-patient-visits-for-xgb-oost-classifier)
 16. Use XGBoost for Time-Series Forecasting - Analytics Vidhya, 访问时间为 二月 11, 2026,
<https://www.analyticsvidhya.com/blog/2024/01/xgboost-for-time-series-forecasting/>
 17. XGBoost Difference Transform Time Series Data, 访问时间为 二月 11, 2026,
<https://xgboosting.com/xgboost-difference-transform-time-series-data/>
 18. 4 Strategies for Multi-Step Time Series Forecasting - MachineLearningMastery.com, 访问时间为 二月 11, 2026,
<https://machinelearningmastery.com/multi-step-time-series-forecasting/>
 19. ML time series forecasting the right way | by Mario Dagrada | TDS Archive - Medium, 访问时间为 二月 11, 2026,
<https://medium.com/data-science/ml-time-series-forecasting-the-right-way-cbf3678845ff>
 20. Direct multi-step forecasting - Skforecast Docs, 访问时间为 二月 11, 2026,
https://skforecast.org/0.16.0/user_guides/direct-multi-step-forecasting.html
 21. XGBoost for Multi-Step Univariate Time Series Forecasting with ..., 访问时间为 二月 11, 2026,
<https://xgboosting.com/xgboost-for-multi-step-univariate-time-series-forecasting-with-multioutputregressor/>
 22. Multistep Forecasting Strategies - Kaggle, 访问时间为 二月 11, 2026,
<https://www.kaggle.com/code/eugeniyosetrov/multistep-forecasting-strategies>
 23. Forecasting With Machine Learning - Kaggle, 访问时间为 二月 11, 2026,
<https://www.kaggle.com/code/georgezoto/forecasting-with-machine-learning>
 24. Multi-output regression with gradient boosting machines | by Krzysztof Joachimiak | Medium, 访问时间为 二月 11, 2026,
<https://medium.com/@joachimiak.krzysztof/multi-output-regression-with-gradient-boosting-machines-39c925b5a1d4>
 25. XGBoost: Multi-Output - Kaggle, 访问时间为 二月 11, 2026,
<https://www.kaggle.com/code/carlolepelaars/xgboost-multi-output>
 26. XGBoost Time Series GridSearchCV with TimeSeriesSplit, 访问时间为 二月 11, 2026,
<https://xgboosting.com/xgboost-time-series-gridsearchcv-with-timeseriessplit/>

27. How To Backtest Machine Learning Models for Time Series Forecasting - MachineLearningMastery.com, 访问时间为 二月 11, 2026,
<https://machinelearningmastery.com/backtest-machine-learning-models-time-series-forecasting/>
28. Time Series Forecasting with XGBoost - Advanced - Kaggle, 访问时间为 二月 11, 2026,
<https://www.kaggle.com/code/youknowjp/time-series-forecasting-with-xgboost-advanced>
29. Choosing model from Walk-Forward CV for Time Series - Cross Validated - Stack Exchange, 访问时间为 二月 11, 2026,
<https://stats.stackexchange.com/questions/440280/choosing-model-from-walk-forward-cv-for-time-series>
30. Ultimate guide to XGBoost library in Python - Deepnote, 访问时间为 二月 11, 2026,
<https://deepnote.com/blog/ultimate-guide-to-xgboost-library-in-python>
31. XGBoost Hyper-parameter Tuning with GPU: The Ultimate Performance Guide - Medium, 访问时间为 二月 11, 2026,
<https://medium.com/@juanc.olamendy/xgboost-hyper-parameter-tuning-with-gpu-the-ultimate-performance-guide-7aadd99f5bc0>
32. Recommendations for tuning XGBoost Hyperparams? - Data Science Stack Exchange, 访问时间为 二月 11, 2026,
<https://datascience.stackexchange.com/questions/108233/recommendations-for-tuning-xgboost-hyperparams>
33. Understanding Monotonic Constraints in XGBoost Models for Improved Interpretability, 访问时间为 二月 11, 2026,
<https://medium.com/@ompanda/understanding-monotonic-constraints-in-xgboost-models-for-improved-interpretability-212e3723cf30>
34. Monotonic Constraints — xgboost 3.2.0-dev documentation, 访问时间为 二月 11, 2026,
<https://xgboost.readthedocs.io/en/stable/tutorials/monotonic.html>
35. Categorical Data — xgboost 3.1.1 documentation, 访问时间为 二月 11, 2026,
<https://xgboost.readthedocs.io/en/stable/tutorials/categorical.html>
36. Categorical Features in XGBoost Without Manual Encoding | NVIDIA Technical Blog, 访问时间为 二月 11, 2026,
<https://developer.nvidia.com/blog/categorical-features-in-xgboost-without-manual-encoding/>
37. How can I reduce processing time with xgboost by utilizing my GPU? - Stack Overflow, 访问时间为 二月 11, 2026,
<https://stackoverflow.com/questions/77643788/how-can-i-reduce-processing-time-with-xgboost-by-utilizing-my-gpu>